西北工业大学实验报告

课程名称:高级语言程序设计实验

实验课次周次:第二次（第八周）

实验地点:实验大楼 B107（软件实验室）

学生姓名：张洛宾

学　　号:2024302696

实验时间:2026-04-25

提交时间:2026-04-30

成绩评定：（满分为 100 分） 指导教师评语： 指导教师签名：

目　录

1　JAVA 实验六 异常处理 2

1.1　实验内容（要求、算法、步骤和方法） 2

1.2　JAVA 源代码文件说明 2

1.3　JAVA 实验结果 2

2　JAVA 实验七 综合实验 7

2.1　实验内容（要求、算法、步骤和方法） 7

2.2　JAVA 源代码文件说明 8

2.3　JAVA 实验结果 8

3　UML 实验一 UML 建模基础 16

3.1　实验内容（要求、算法、步骤和方法） 16

3.2　UML 结果文件说明 16

3.3　UML 实验结果 16

4　实验二 UML 用例图 18

4.1　实验内容（要求、算法、步骤和方法） 18

4.2　UML 结果文件说明 19

4.3　UML 实验结果 19

5　实验总结 23

5.1　实验中遇到的问题以及解决方法 23

5.2　实验体会 23

1 Java 实验六 异常处理

1.1 实验内容（要求、算法、步骤和方法）

编写一个程序，同时捕获数组越界和被 0 除的异常，说明异常处理语句 try-catch-finally 的处理机制。

练习内容：创建用户自定义异常，用于描述数据取值范围的错误信息。

思考问题：

（1） 本程序中 throws 和 throw 语句的作用是什么？

（2） 本程序中是如何定义用户自定义异常的？

（3） 本程序是如何处理程序产生的用户自定义异常的？

（4） 如果将程序中的“public void regist(int num) throws MyException”改为“public void regist(int num) ”，会出现什么样的情况？

上机作业：编写一个程序，将字符串转换成数字。请使用 try-catch-finally 语句处理转换过程中可能出现的异常。

1.2 Java 源代码文件说明

具体代码请参见附件中电子版的 java 源程序。源程序的简要说明如下：

CatchDemo.java —— 实验指导书中的第一个内容，使用 try-catch-finally 来捕获基本异常.

TestException.java —— 实验指导书中的练习内容,创建用户自定义异常(继承自 Exception),使用程序主动抛出自定义异常并捕获..

String2Number.java —— 实验指导书中上机作业，使用 try-catch-finally 来实现字符串转换为数字的异常处理.

1.3 Java 实验结果

(包括重点代码说明，思路，运行结果，以及实验要求中需要书面回答的内容)

```
public class  CatchDemo {
    public static void main(String[] args) {
        int number[]={4,8,16,32,64,128,256,512};
        int denom[]={2,0,4,4,0,8};
```

```
        for(int i=0;i<number.length ;i++){
            try {
                System.out.println(number[i] + " / " + denom[i] + "is "
+
                        number[i] / denom[i]);
            }catch(ArithmeticException exc){
                System.out.println("Can't be divided be zero");
            }
            catch(ArrayIndexOutOfBoundsException exc){
                System.out.println("No matching element found.");
            }
            finally{
                System.out.println("Finally 已执行");
            }
        }

    }

}
```

代码说明

主函数部分,利用 try-catch-finally 语句来捕获可能出现的基本的异常,如 ArithmeticException 和 ArrayIndexOutOfBoundsException,一个用于处理除数为 0 的情况,一个用于处理数组越界的情况。

思路

简单使用 try-catch-finally 处理即可。运行结果如下:

```
4 / 2is 2
Finally已执行
Can't be divided be zero
Finally已执行
16 / 4is 4
Finally已执行
32 / 4is 8
Finally已执行
Can't be divided be zero
Finally已执行
128 / 8is 16
Finally已执行
No matching element found.
Finally已执行
No matching element found.
Finally已执行
```

如果没有异常处理,程序会在第一个报错的位置停止运行,并且抛出异常 Exception in thread "main" java.lang.ArithmeticException: / by zero at test11.CatchDemo.main(CatchDemo.java:12)

练习内容：

源代码如下：

```
class UserException extends Exception
{
    private int idnumber;

    public UserException(String message, int id)
    {
        super(message);
        this.idnumber = id;
    }

    public int getId()
    {
        return idnumber;
    }
}//自定义异常类

public void regist(int num) throws UserException
{
    if (num < 0)
```

```
    {
        throw new UserException("人数为负值，不合理", 3);
    }
    System.out.println("登记人数：" + num);
}

public void manager()
{
    try
    {
        regist(-100);
    }
    catch (UserException e)
    {
        System.out.println("登记出错，类别：" + e.getId());
    }
    System.out.println("本次登记操作结束");
}
```

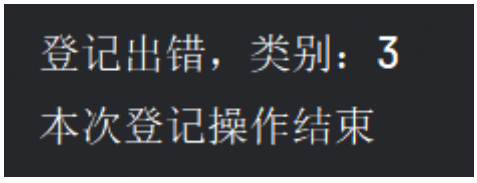
代码说明

UserException 继承了 java 中的 Exception 类,并且完成了父类的初始化,以及得到错误 ID.函数 regist 可以主动抛出异常 UserException,然后再函数 manager 中调用 regist 并且使用 try-catch 来捕获自定义的异常。

思路

通过继承 Exception 类,来实现自定义的异常类,并且在 regist 函数的声明的过程中说明可能会抛出异常,然后在函数体的内部通过代码检查主动抛出异常,然后在调用体当中使用 try-catch 来保证程序的鲁棒性。

运行结果如下



思考问题

(1. throws 用于创建函数或者方法的时候,声明该方法可能会抛出异常,throw 用于在函数或者方法当中主动抛出异常.

(2. 本程序创建了 UserException 通过继承 Exception 类来实现自定义的异常,其中有错误 ID,在构造方法当中利用 message 来对父类进行构造,同时也写了 getID 的方法,

实现了一个异常类应该有的方法和属性.

(3. 本程序在 regist 函数中主动抛出异常,在 manager 函数当中调用 regist 函数,使用 try 来调用函数,catch 来捕获异常并处理.

(4. 首先程序会出现编译错误,因为语句中有关于异常的处理.

上机作业：

源代码如下：

```
try
{
    if (!str.matches("^[+-]?\\d+$"))
    {
        throw new NumberFormatException("字符串包含非数字字符");
    }
    int intValue = Integer.parseInt(str);
    System.out.println("成功转为整数: " + intValue);
}
catch (NumberFormatException e)
{
    System.out.println("整数转换失败: " + e.getMessage());
    try
    {
        if
(!str.matches("^[+-]?(\\d+\\.?\\d*|\\.\\d+)([eE][+-]?\\d+)?$"))
        {
            throw new NumberFormatException("字符串不是有效的数字格式");
        }
        double doubleValue = Double.parseDouble(str);
        System.out.println("成功转为浮点数: " + doubleValue);
    }
    catch (NumberFormatException e2)
    {
        System.out.println("浮点数转换也失败: " + e2.getMessage());
    }
}
finally
{
    System.out.println("转换尝试结束\n");
}
```

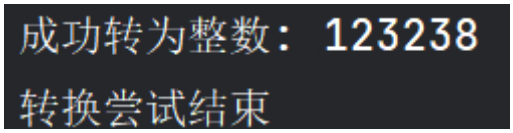
代码说明

使用嵌套的 try-catch 来实现字符串转换为整数或者浮点数,利用 Java 中内置的 NumberFormatException 异常,和类的转换静态方法来实现转换,然后利用 str.match() 来进行字符串的初步判断。

思路

使用 Java 内置的 NumberFormatException 来捕获异常,利用 java 内置的转换方法,利用正则表达式来初步判断字符串是否只包含数字,最后用 try-catch 实现整体结构。

运行结果如下：



2 Java 实验七 综合实验

2.1 实验内容（要求、算法、步骤和方法）

题目 1. 学生信息管理系统

1）基本需求：

实现一个简单的学生信息管理系统，该系统能够完成以下功能：

【1.】 添加新学生信息；

【2.】 根据学号查询学生信息；

【3.】 根据学号删除学生信息；

【4.】 列出系统中的所有学生信息；

【5.】 将学生信息持久化保存到磁盘文件中；

【6.】 从磁盘文件中读取学生信息至系统。

2）数据要求：

学生信息包括但不限于：

学号（唯一标识符）

姓名

出生日期

性别

班级

3） 详细说明：

【1.】 设计一个学生类（Student），包含上述基本数据域与方法；

【2.】 设计一个学生管理类（StudentManager），用于实现功能【1】-【4】；

【3.】 学生管理类应该有集合成员，选择合适的集合类型来存储学生对象；

【4.】 在学生管理类中实现数据的存取方法 saveToFile() 和 loadFromFile()，将集合中的数据序列化到文件，从文件反序列化到集合；

【5.】 主类（StudentMngMain）中实现用户交互，通过命令行输入指令，调用 StudentManager 类的方法完成对应的操作；

【6.】 所有的类都应该在合适的地方处理异常，如文件未找到、输入输出异常等；

【7.】 对输入数据进行合理的验证，确保系统的健壮性

2.2 Java 源代码文件说明

具体代码请参见附件中电子版的 java 源程序。源程序的简要说明如下：

StudentManager.java —— 提供 Main 函数中的方法实现，以及组织学生集合。

2.3 Java 实验结果

（包括重点代码说明，思路，运行结果，以及实验要求中需要书面回答的内容）

源代码如下：

```
import java.io.*;
import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.*;

// 学生类，实现 Serializable 以支持对象序列化
class Student implements Serializable {
    private static final long serialVersionUID = 1L; // 序列化版本号

    private String studentId;   // 学号
    private String name;        // 姓名
    private Date birthDate;     // 出生日期
    private String gender;      // 性别
```

```
    private String className;   // 班级

    public Student(String studentId, String name, Date birthDate, String
gender, String className) {
        this.studentId = studentId;
        this.name = name;
        this.birthDate = birthDate;
        this.gender = gender;
        this.className = className;
    }

    // Getter 和 Setter 方法
    public String getStudentId() { return studentId; }
    public void setStudentId(String studentId) { this.studentId =
studentId; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public Date getBirthDate() { return birthDate; }
    public void setBirthDate(Date birthDate) { this.birthDate =
birthDate; }
    public String getGender() { return gender; }
    public void setGender(String gender) { this.gender = gender; }
    public String getClassName() { return className; }
    public void setClassName(String className) { this.className =
className; }

    @Override
    public String toString() {
        SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
        return String.format("学号: %-10s 姓名: %-8s 性别: %-4s 出生日
期: %-10s 班级: %-10s",
                studentId, name, gender, sdf.format(birthDate),
className);
    }
}

// 学生管理类，负责增删查改及持久化
class StudentManager {
    private Map<String, Student> studentMap; // 使用 HashMap 存储学生，
学号为键
```

```
    private static final String DATA_FILE = "students.dat"; // 默认数据文件

    public StudentManager() {
        studentMap = new LinkedHashMap<>(); // 保持插入顺序
    }

    // 1. 添加学生信息
    public boolean addStudent(Student student) {
        if (studentMap.containsKey(student.getStudentId())) {
            System.out.println("错误：学号 " + student.getStudentId() + " 已存在，添加失败。");
            return false;
        }
        studentMap.put(student.getStudentId(), student);
        System.out.println("学生 " + student.getName() + " 添加成功。");
        return true;
    }

    // 2. 根据学号查询学生信息
    public Student queryStudent(String studentId) {
        Student student = studentMap.get(studentId);
        if (student == null) {
            System.out.println("未找到学号为 " + studentId + " 的学生。");
        }
        return student;
    }

    // 3. 根据学号删除学生信息
    public boolean deleteStudent(String studentId) {
        if (studentMap.remove(studentId) != null) {
            System.out.println("学号 " + studentId + " 的学生已删除。");
            return true;
        } else {
            System.out.println("错误：学号 " + studentId + " 不存在，删除失败。");
            return false;
        }
    }
```

```
    // 4. 列出所有学生信息
    public void listAllStudents() {
        if (studentMap.isEmpty()) {
            System.out.println("系统中暂无学生信息。");
            return;
        }
        System.out.println("\n========== 学生信息列表 ==========");
        for (Student student : studentMap.values()) {
            System.out.println(student);
        }
        System.out.println("共 " + studentMap.size() + " 名学生。\n");
    }

    // 5. 保存数据到文件（序列化）
    public void saveToFile() {
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream(DATA_FILE))) {
            oos.writeObject(new ArrayList<>(studentMap.values())); //
保存学生列表
            System.out.println("学生信息已保存到文件 " + DATA_FILE);
        } catch (IOException e) {
            System.err.println("保存文件时出错: " + e.getMessage());
        }
    }

    // 6. 从文件加载数据（反序列化）
    @SuppressWarnings("unchecked")
    public void loadFromFile() {
        File file = new File(DATA_FILE);
        if (!file.exists()) {
            System.out.println("数据文件不存在，将使用空集合启动。");
            return;
        }
        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream(DATA_FILE))) {
            List<Student> list = (List<Student>) ois.readObject();
            studentMap.clear();
            for (Student s : list) {
                studentMap.put(s.getStudentId(), s);
```

```
            }
            System.out.println("已从文件 " + DATA_FILE + " 加载 " +
studentMap.size() + " 条学生记录。");
        } catch (IOException | ClassNotFoundException e) {
            System.err.println("加载文件时出错: " + e.getMessage());
        }
    }
}

// 主类，命令行交互
public class StudentMngMain {
    private static Scanner scanner = new Scanner(System.in);
    private static StudentManager manager = new StudentManager();

    public static void main(String[] args) {
        // 启动时加载已有数据
        manager.loadFromFile();

        while (true) {
            printMenu();
            String choice = scanner.nextLine().trim();
            switch (choice) {
                case "1":
                    addStudent();
                    break;
                case "2":
                    queryStudent();
                    break;
                case "3":
                    deleteStudent();
                    break;
                case "4":
                    manager.listAllStudents();
                    break;
                case "5":
                    manager.saveToFile();
                    break;
                case "6":
                    manager.loadFromFile();
                    break;
```

```
                case "0":
                    System.out.println("系统退出。");
                    manager.saveToFile(); // 退出时自动保存
                    scanner.close();
                    System.exit(0);
                    break;
                default:
                    System.out.println("无效选项，请重新输入。");
            }
        }
    }

    // 打印菜单
    private static void printMenu() {
        System.out.println("\n========== 学生信息管理系统
==========");
        System.out.println("1. 添加学生");
        System.out.println("2. 按学号查询学生");
        System.out.println("3. 按学号删除学生");
        System.out.println("4. 显示所有学生");
        System.out.println("5. 保存数据到文件");
        System.out.println("6. 从文件加载数据");
        System.out.println("0. 退出系统");
        System.out.print("请输入选项：");
    }

    // 添加学生时的输入验证和交互
    private static void addStudent() {
        try {
            System.out.print("请输入学号：");
            String id = scanner.nextLine().trim();
            if (id.isEmpty()) {
                System.out.println("学号不能为空！");
                return;
            }

            System.out.print("请输入姓名：");
            String name = scanner.nextLine().trim();
            if (name.isEmpty()) {
                System.out.println("姓名不能为空！");
```

```
                return;
            }

            System.out.print("请输入出生日期（格式 yyyy-MM-dd）: ");
            String dateStr = scanner.nextLine().trim();
            Date birthDate = parseDate(dateStr);

            System.out.print("请输入性别（男/女）: ");
            String gender = scanner.nextLine().trim();
            if (!gender.equals("男") && !gender.equals("女")) {
                System.out.println("性别只能为 '男' 或 '女'! ");
                return;
            }

            System.out.print("请输入班级: ");
            String className = scanner.nextLine().trim();
            if (className.isEmpty()) {
                System.out.println("班级不能为空! ");
                return;
            }

            Student student = new Student(id, name, birthDate, gender,
classNname);
            manager.addStudent(student);
        } catch (ParseException e) {
            System.out.println("日期格式错误，请使用 yyyy-MM-dd 格式。");
        } catch (Exception e) {
            System.out.println("输入错误: " + e.getMessage());
        }
    }

    // 查询学生
    private static void queryStudent() {
        System.out.print("请输入要查询的学号: ");
        String id = scanner.nextLine().trim();
        Student student = manager.queryStudent(id);
        if (student != null) {
            System.out.println("查询结果: " + student);
        }
    }
```

```
    // 删除学生
    private static void deleteStudent() {
        System.out.print("请输入要删除的学号: ");
        String id = scanner.nextLine().trim();
        manager.deleteStudent(id);
    }

    // 解析日期，格式要求 yyyy-MM-dd
    private static Date parseDate(String dateStr) throws ParseException
{
        SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
        sdf.setLenient(false); // 严格校验日期合法性
        return sdf.parse(dateStr);
    }
}
```

代码说明

该程序是一个命令行交互的学生信息管理系统，包含 Student 实体类（实现序列化，存储学号、姓名、出生日期、性别、班级）、StudentManager 管理类（用 LinkedHashMap 维护学生记录，提供增删查改及基于文件 students.dat 的序列化持久化操作）和主类 StudentMngMain（提供菜单循环、输入校验及调用管理方法），支持添加、按学号查询删除、列出全部学生、手动保存和加载数据，并在退出时自动保存，确保数据持久化。

思路

整体的思路很简单，首先构造了单个学生的信息集合与方法集合，就是 Student 类，其实现了私有方法以及其 setter 和 getter 方法，体现了 Java 中的封装的思想，然后实现 Serializable 接口，提供文件化存储能力。

基于 Student 类，构建了 StudentManager 类，利用 ArrayList 组织 Student 类，然后实现主体的增删改查功能以及文件交互功能，还有输入验证功能，让程序更加健壮.

最有在 Main 类，负责组织 StudentManager 类提供的功能，和提供界面。

运行结果如下：

```
========== 学生信息管理系统 ==========
1. 添加学生
2. 按学号查询学生
3. 按学号删除学生
4. 显示所有学生
5. 保存数据到文件
6. 从文件加载数据
0. 退出系统
请输入选项：1
请输入学号：2024302696
请输入姓名：robin
请输入出生日期（格式 yyyy-MM-dd）：2006-03-09
请输入性别（男/女）：男
请输入班级：10012404
学生 robin 添加成功。
```

UML 实验一 UML 建模基础

2.4 实验内容（要求、算法、步骤和方法）

1、练习使用建模工具建立各种 UML 图形，并对图形进行相应编辑和修改。

2、 认识各种 UML 关系及可见性符号，并用工具表示出来。

分析与讨论:总结 UML 在软件工程中的作用以及使用 UML 建模的必要性。

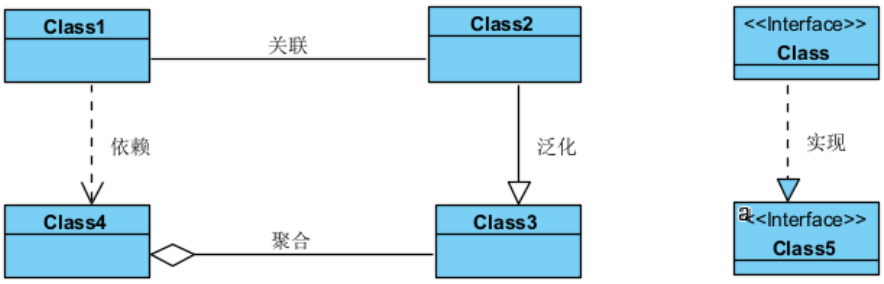
2.5 UML 结果文件说明

具体代码请参见附件中电子版的文件。结果文件的简要说明如下：

UML01&02.vpp ---- 主要是对 UML 建模软件的操作进行熟悉

2.6 UML 实验结果

(包括绘制的 UML 图，UML 图的说明，以及实验要求中需要书面回答的内容)



说明

本次实验主要是熟悉 UML 建模的操作以及实现类之间的各种关系和属性与类的可见性的修改,如上图分别实现了关联关系,依赖关系,泛化关系和实现关系,以及聚合关系.同时class2当中有设置属性的可见性.如图book是字符串类型,并且设置为Package类型,下面的 stident 属性被设置为 public。

分析与讨论：

UML（统一建模语言）在软件工程中扮演着重要角色，它通过标准化的图形符号和规范为软件开发提供了一套通用的可视化建模工具，能够清晰地描述系统的结构、行为和交互。UML 不仅帮助开发团队统一沟通语言，减少歧义，还能辅助需求分析、系统设计、代码实现和文档生成，从而提高开发效率和质量。此外，UML 支持从不同视角（如静态结构、动态行为）对系统进行抽象，使得复杂系统的理解和维护变得更加直观和高效。

使用 UML 建模的必要性在于它能够将抽象的软件设计转化为可视化的模型，从而在早期阶段发现潜在问题，降低开发风险。通过 UML 图（如类图、用例图、时序图等），团队可以更准确地捕捉用户需求，规划系统架构，并确保设计与实现的一致性。建模过程还能促进团队成员之间的协作，为后续的代码编写、测试和维护提供明确的参考依据。尤其在面对复杂系统或大型项目时，UML 建模成为不可或缺的桥梁，连接了需求、设计与实现，最终提升软件的可扩展性和可维护性。

3 实验二 UML 用例图

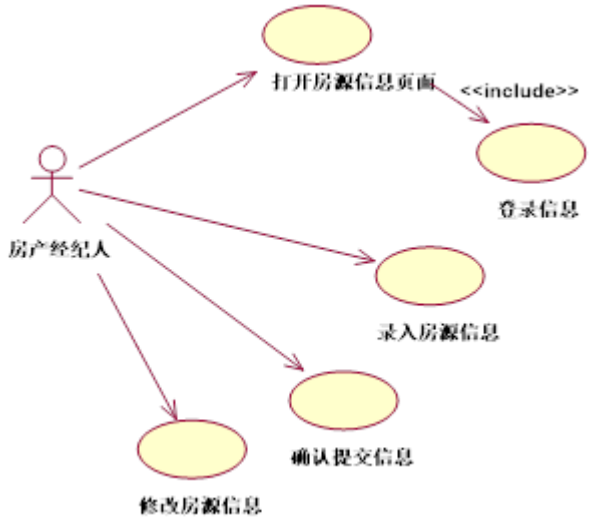
3.1 实验内容（要求、算法、步骤和方法）

1、 什么是用例，什么是场景？用例和场景之间的关系是怎样的？

2、 用例图中有哪些组成元素？在 UML 中是如何表示的？

3、 用例与用例之间的包含关系、扩展关系和泛化关系各代表什么含义？它们之间有何区别？对以上三种关系各举一例，画出用例图，并进行说明。

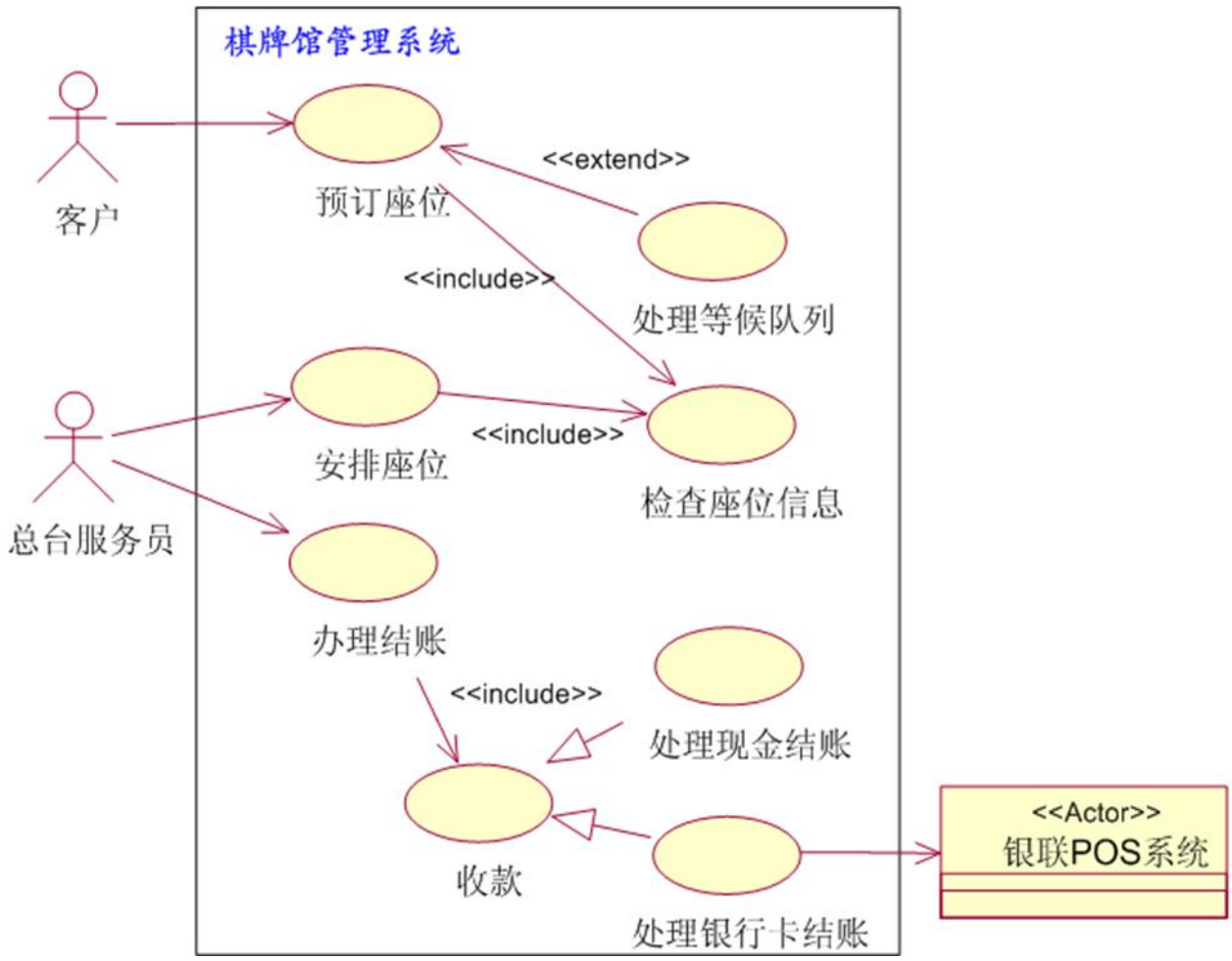
4、 为了满足物业中介行业的信息化要求，甲公司基于详尽的需求调研与分析，准备研发一套符合市场需要的、实用的信息管理系统。主要将实现客户资料信息管理、客户委托（出租、出售、租赁、购买）信息管理、业务线索生成与管理、房源状态自动更新、权限管理、到期用户管理、房源组合查询等功能。该公司小王，通过多次的与潜在客户的交流与沟通，完成了最初的用例模型的开发，下是一个用例模型的局部：



（1） 但小李认为该模型不符合“用例建模”的思想，存在明显的错误。说明错误所在，并说明应该如何修改。

（2） 在上图中构造型“《include》”表示的是什么意思，它与“《extend》”之间的区别是什么？

5、 阅读下面的用例图，说明该图所表达的信息。



3.2 UML 结果文件说明

具体代码请参见附件中电子版的文件。结果文件的简要说明如下：

UML01&02.vpp ---- 第一个实验与第二个实验写在一起,这个是用例图的部分

3.3 UML 实验结果

(包括绘制的 UML 图，UML 图的说明，以及实验要求中需要书面回答的内容)

问题 1

用例（Use Case）是 UML 中用于描述系统功能的一种建模方法，它从用户的角度定义系统应该完成的任务或目标。用例不关心具体实现细节，而是关注 Actor 使用系统完成说明功能.

场景是一个用例的具体执行实例,用于描述特定条件下,用户与系统交互

的步骤,一个用例一般包含多个场景.

用例是场景的功能描述,场景是用例的具体执行流程.一个用例有多个场景。

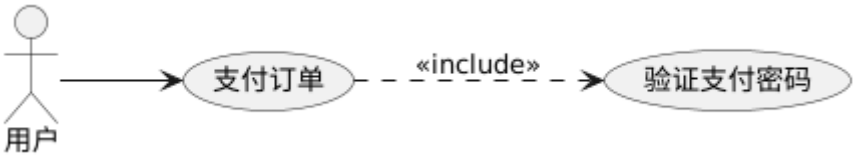
问题 2

包含参与者(Actor),用例(Use Case),关系(包括关联,包含,拓展,泛化四种关系),系统边界(界定系统范围)这四个要素。

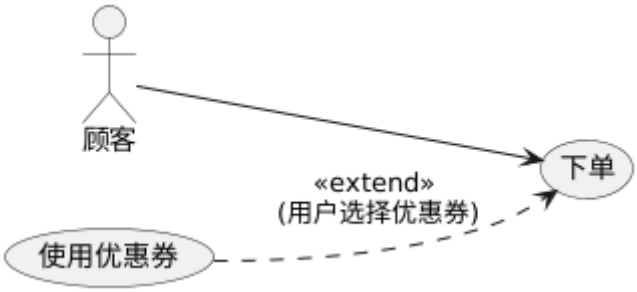
在 UML 当中,Actor 用小人表示,Use Case 用椭圆形表示,内部标注用例名称.Relationships 用箭头表示(Association 用实线箭头,include 用虚线箭头,并且标注<<include>>,Extend 用虚线箭头表示,标注<<extend>>,并且注明拓展条件.系统边界用矩形框表示.还有泛化使用空心箭头表示。

问题 3

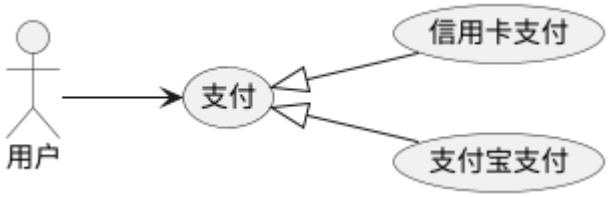
Include 表示一个用例必须调用另一个用例的功能，属于强依赖关系。无法独立存在。



Extend 表示一个用例在特定条件下扩展另一个用例的功能，属于可选依赖关系。扩展用例是附加功能，只在满足条件时触发。



Generalization 表示用例之间的继承关系，子用例继承父用例的行为并可以扩展或覆盖。类似面向对象中的“父类与子类”，子用例是父用例的特化版本。



问题 4

(1) 错误的将系统操作步骤作为用例,用例图应该描述的是参与者如何与系统进行交互,而不是具体的实现步骤.登陆信息,打开房源界面等都是具体操作步骤而不是用例.修改方法是将打开房源界面,录入房源信息,修改房源信息这三个整合成一个用例,即为管理房源信息。

包含关系的滥用,打开房源信息界面和登陆信息之间的 include 关系不合理.重构包含关系,将登陆独立出来,需要验证信息的就利用 include 关系。

缺失一部分关键功能,比如组合查询,房源状态自动更新等功能.添加对应功能。

(2) Include 表示一个用例必须调用另一个用例的功能，属于强依赖关系。无法独立存在. Extend 表示一个用例在特定条件下扩展另一个用例的功能，属于可选依赖关系。扩展用例是附加功能，只在满足条件时触发。

问题 5

该棋牌馆管理系统允许客户预订座位（需排队时自动处理等候队列），总台服务员负责安排已验证状态的座位，并提供结账服务（支持现金或银行卡支付，银行卡支付需通过银联 POS 系统完成）。系统通过清晰的用例划分实现了从预订到结算的全流程管理。

分析与讨论：

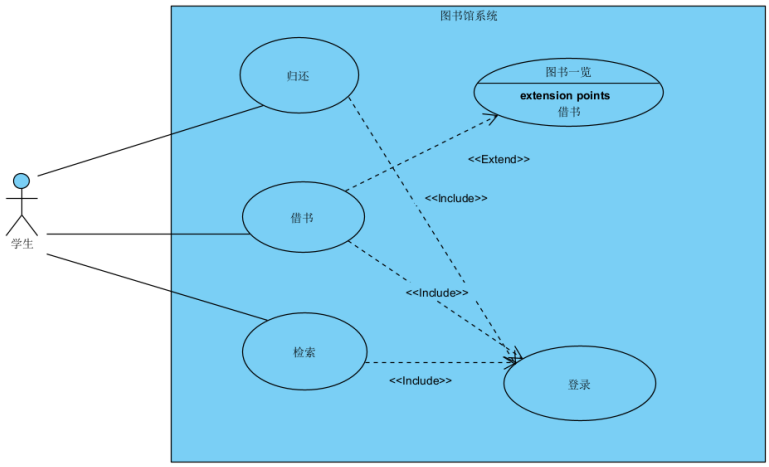
(1. 用例图的核心作用是直观呈现系统功能边界、参与者与用例的交互关系，它通过标准化的图形符号（如椭圆表用例、人形表参与者）帮助团队在需求分析阶段快速达成共识，明确"系统做什么"和"谁在使用什么功能"。使用场合包括需求分析,文档编写,项目评审.

(2. 用例分析技术和结构化分析都是需求工程中的重要方法，但侧重点不同：用例分析以用户视角为核心，通过参与者-系统交互场景（如"预订座位"）描述功能需求，强调行为逻辑和用户目标，适用于面向对象系统设计；而结构化分析则采用数据流图（DFD）和过程分解，聚焦数据加工流程（如"座位信息验证"），更适合传统结构化系统开发。二者本质区别在于——用例分析是外部驱动（回答"系统为谁提供什

么服务"），结构化分析是内部驱动（回答"数据如何被处理"），实际项目中常结合使用，例如先用用例图划定系统边界，再用 DFD 细化内部数据处理流程。

(3. 在划分用例时，应遵循"用户目标导向"原则，每个用例代表一个完整的、对参与者有价值的业务功能（如"借阅图书"而非"输入书号"）。关键注意事项包括：1）保持用例粒度适中——既不过于宽泛（如"管理图书馆"），也不过度碎片化（如"点击查询按钮"）；2）严格区分核心流程与异常分支，通过«extend»关系处理可选场景（如"逾期还书"）；3）避免功能重叠，若多个用例共享步骤（如"用户认证"），用«include»抽取为子用例；4）参与者必须来自系统外部（如"读者"或"图书管理员"），不可将系统内部模块作为参与者。例如在图书馆系统中，"检索图书"和"办理借阅"应作为独立用例，而"验证读者身份"可作为被包含用例复用。

实验内容



该 UML 用例图实现了一个图书馆的系统,包括借书还书检索书等功能,同时也有信息安全系统(信息认证),要求每个功能都必须强制调用信息认证,这通过 include 关系来实现.同时还有查看已经借的书,这通过 extend 来实现。

4 实验总结

4.1 实验中遇到的问题以及解决方法

问题 1： 在选择 Student 类的组织形式的时候,遇到了诸多的选择,比如自定义的链表,或者 ArrayList 集合或者或者数组。

解决：最后权衡之下选择了 ArrayList 集合,因为有良好的方法和性能实现,不需要自己去写方法。

问题 2：在实现 StudentManager 方法的时候,遇到了如何与 Main 类进行交互并且实现输入检查的功能的问题。

解决：最后统一在 StudentManager 类当中定义了一个验证输入的方法,然后再 Main 类当中调用这种方法,保证 Java 的封装特性与程序的健壮性。

问题 3：在使用 Visual Paradigm 软件的时候,不明白项目的组成是什么,在项目中同时创建了类图和用例图,但是不会切换。

解决：经过搜索之后,发现应该在检视里面的项目浏览器中切换同一个项目下的不同的图。

问题 4：在创建类图的过程中,不会修改类的属性和类本身的访问权限。

解决：经过搜索之后,发现在类图的右上角有一个图标点击之后就可以设置访问权限了,还能添加属性和方法。

4.2 实验体会

通过这次实验，我收获了很多关于 Java 编程和 UML 建模的实践经验。在 Java 实验部分，我掌握了异常处理的基本方法，学会了如何使用 try-catch-finally 结构来增强程序的健壮性。通过自定义异常类，我理解了异常机制的灵活性。特别是在综合实验中，通过实现学生管理系统，我体会到了封装的重要性——将数据和方法组织在类中，再通过接口进行交互，这种设计思想让我很受启发。

UML 实验让我认识到可视化建模工具的价值。刚开始画类图时，我经常混淆关联和依赖关系，通过反复练习才逐渐掌握。用例图的分析过程也让我明白：好的系统设计要从用户角度出发，而不是单纯考虑技术实现。

这次实验也暴露出我的不足：对集合框架的使用还不够熟练，在 ArrayList 操作时经常要查阅文档；对 UML 关系的理解还需要加强。建议未来的实验可以增加更多实际案例，比如结合数据库的小项目，这样能更好地锻炼综合能力。

总的来说，这些实验不仅巩固了课堂知识，更让我体会到"纸上得来终觉浅"的道理。作为初学者，我会继续练习编码规范，培养调试能力，为后续学习打下坚实基础。